

Æ

A L E J A N D R A E S T E O

Bitácora de Arreglos florales Ikenobo

Estos últimos días fueron bastante intensos :|.

Si bien ya había logrado hacer funcionar el proyecto en Java con consola y listas en memoria, pasar todo eso a una aplicación web con Spring Boot fue un cambio importante. Sentí que tenía que reaprender la forma de pensar el programa.

Casi dejo el curso!!!

Lo primero que tuve que entender fue que ya no existía ese recorrido lineal del método main ejecutando todo paso a paso. Ahora la aplicación queda corriendo en segundo plano, esperando solicitudes y respondiendo según el endpoint que se invoque. Empecé a familiarizarme con el uso de `@RestController`, los métodos HTTP (GET, POST, PUT y DELETE) y con la forma en que el frontend y el backend empiezan a "conversar" entre sí.

El puente de datos, Java y el parsing de JSON

Uno de los desafíos más grandes fue conectar el panel de administración con el backend. Al principio estaba convencida de que enviaba los datos correctamente, pero el backend no pensaba lo mismo. Descubrí que Java es un lenguaje bastante estricto y no perdona cuando la estructura del JSON que recibe no coincide exactamente con el tipo de datos de las entidades.

En mi caso, enviaba la categoría como un texto plano cuando en realidad el modelo requería un objeto Categoría. Cuando entendí cómo Spring Boot y la librería Jackson convierten automáticamente ese JSON en objetos de Java, muchas piezas del rompecabezas empezaron a tener sentido.

Hibernate, MySQL y el lore de los IDs

Después apareció uno de esos errores que parecen no terminar nunca: una respuesta HTTP 500 provocada por una restricción de clave foránea en la base de

datos. Me costó bastante encontrar el origen porque el código Java parecía estar bien y, la verdad, no encontraba el error...

Al inspeccionar directamente la base de datos MySQL descubrí que los IDs de las categorías ya no eran los que yo suponía. Entre pruebas, reinicios de la aplicación y registros generados por Hibernate, los identificadores autoincrementales habían cambiado. El backend intentaba relacionar nuevos productos con categorías cuyos IDs ya no existían en el estado actual de la tabla. Una vez corregidos esos valores, todo volvió a funcionar y entendí mucho mejor cómo trabajan Hibernate y las relaciones @ManyToOne entre tablas.

Expansión del sistema y refactorización

Mientras iba resolviendo la integración, también fui agregando nuevas funcionalidades y reorganizando el proyecto.

Implementé la búsqueda de productos tanto por nombre como por categoría, empecé a desarrollar la lógica del carrito de compras, incorporé validaciones utilizando las anotaciones de Jakarta Validation y seguí ordenando la arquitectura, separando correctamente controladores, servicios y repositorios a medida que el proyecto iba creciendo.

En más de una oportunidad aparecieron errores de Maven, dependencias que dejaban de resolverse, clases que de un momento a otro parecían desaparecer del proyecto o problemas con el reconocimiento del workspace en VS Code. Fueron horas de prueba y error, pero también sirvieron para entender mejor cómo está organizado internamente un proyecto de Spring Boot.

Gestión de imágenes

Otra tarea que me llevó bastante tiempo fue decidir cómo manejar las imágenes de los productos. Estuve evaluando distintas alternativas para alojarlas de forma externa, comparando opciones como Fake Store API, Google Drive y Cloudinary. Finalmente terminé inclinándome por utilizar URLs externas para mantener el proyecto liviano y evitar almacenar archivos dentro del repositorio.

También tuve que adaptar el modelo Producto, agregar el atributo imageUrl, modificar constructores, servicios y la carga inicial de datos para que todo siguiera

funcionando correctamente sin afectar la relación con las categorías. La IA de VCS no me la hizo fácil, me dejé llevar de su mano y terminé borrando la base de datos, porque los links de las imágenes los autocorregía y terminaba agregando caracteres, por lo tanto las imágenes nunca se veían :(. Ahí me ayudaron las otras IA!!!!

Aprendizaje "detrás de escena"

Si miro todo el recorrido, creo que el mayor aprendizaje no fue solamente incorporar nuevas herramientas o frameworks, sino aprender a investigar cuando algo no funciona. Muchas veces pasé más tiempo leyendo logs, revisando la base de datos, siguiendo el recorrido de una petición HTTP o buscando el origen de una excepción que escribiendo código nuevo.

Con el tiempo empecé a entender mucho mejor qué sucede detrás de escena cada vez que el navegador hace una petición, cómo Spring la recibe, cómo Hibernate consulta la base de datos y cómo finalmente la respuesta vuelve al frontend.

Todavía quedan cosas por terminar, especialmente el carrito de compras y algunos detalles de integración, pero siento que el proyecto ya dejó de ser un conjunto de archivos sueltos para convertirse en una aplicación Full Stack con una arquitectura mucho más sólida, creo, vos me darás tu sabia devolución obviamente. Y, aunque Java todavía me hace enloquecer de vez en cuando, ya no me intimida como al principio de la cursada :).

El proyecto quedó subido a GitHub listo para ser evaluado. Si bien el historial de commits no refleja cada pequeño cambio porque lo subí al final de hacer todos los cambios que necesitaba (y me faltan muchos, pero la entrega es ahora y no en enero), detrás de ese repositorio hay muchas horas de pruebas, errores y demases hasta lograr que el frontend y el backend funcionen juntos de forma consistente.

Continuará...

Y obviamente: GRACIAS TOTALES, MIGUE!!!!!!